

Kod Çoklu Sunucu Uyumluluk Raporu

3-4 API, Redis HA ve PostgreSQL HA hedefi için zorunlu uygulama değişiklikleri

NET SONUÇ

bf92edc3 sürümü mevcut haliyle çoklu API üretim kullanımına uygun değildir. Device attestation state'i, SignalR, Data Protection, yerel dosyalar ve hosted worker sahiplenmesi düzeltilmeden load balancer arkasında birden fazla API aktif edilmemelidir.

Alan	Değer
İncelenen sürüm	Git commit bf92edc3
Kapsam	ASP.NET Core API/WebUI, Redis, SignalR, PostgreSQL, worker, medya
Hedef	Stateless ve aktif-aktif API katmanı
Sürüm	1.0 - 29 Ağustos 2026

Bu rapor, 4.000 eşzamanlı aktif kullanıcı hedefi için başlangıç mimarisini tanımlar. Eşzamanlı kullanıcı sayısı tek başına RPS, sorgu maliyeti veya CPU ihtiyacını kesinleştirmez. Bu nedenle verilen kaynaklar üretime giriş tabanıdır; nihai kapasite k6/JMeter yük testi ve gerçek APM ölçümleriyle doğrulanmalıdır.

1. Yönetici özeti

Öncelik	Bulgu	Üretim etkisi	Karar
P0	Device secret/challenge/nonce IMemoryCache	Rastgele 401 ve replay koruması atlatma	Redis state store zorunlu
P0	SignalR backplane yok	Node'lar arası mesaj ve grup kaybı	Redis backplane zorunlu
P0	Data Protection local key ring	Credentials diğer node'da çözülemez	Ortak key ring zorunlu
P0	Upload/feed local disk	Node'a göre 404/kayıp çıktı	Ortak object/media storage zorunlu
P0	Hosted worker her API'de	Çift e-posta, kargo, tracking, hakediş	Atomic claim + distributed lock zorunlu
P1	Login/rate limit process-local	Limit üç-dört katına çıkar	Redis atomik sayaç veya Nginx ana limit
P1	Tek-host PostgreSQL datasource	Promotion sonrası otomatik primary seçimi yok	Npgsql multi-host datasource

Release kapısı

P0 maddeler tamamlanıp iki API ile çapraz-node entegrasyon testleri geçmeden production load balancing açılmamalıdır.

2. P0-1 - Device state Redis'e taşınmalı

Kesin kanıt

Dosya	Satır	Mevcut davranış
Services/Store/DeviceAttestationServices.cs	195-222	Device signing secret IMemoryCache'e yazılıyor ve oradan okunuyor.
Controllers/StoreDeviceController.cs	15-48	Attestation challenge IMemoryCache'e yazılıyor ve process içinde siliniyor.
Middleware/DeviceRequestGuardMiddleware.cs	68-94	Replay nonce kontrolü IMemoryCache üzerinde.

Gerekli uygulama tasarımı

- IDeviceStateStore isimli abstraction oluşturun: StoreChallengeAsync, ConsumeChallengeAsync, StoreSecretAsync, GetSecretAsync, TryConsumeNonceAsync.
- Challenge tüketimi ve nonce tüketimi Redis Lua veya SET key value NX EX ile atomik olmalı. GET ardından DEL şeklindeki iki ayrı çağrı kabul edilmez.
- Anahtarlar ECSPros:state:device:* prefix'iyle tenant/type ayrımlı tutulmalı; TTL mevcut token/challenge sürelerinden alınmalı.
- Redis hata verdiğinde güvenlik akışı fail-closed çalışmalı; attestation/replay kontrolü cache yokmuş gibi devam etmemeli.

Kabul testi

- Challenge API-1'den alınır, attest API-2'ye gönderilir ve bir kez başarılı olur.
- Aynı challenge API-3'e tekrar gönderildiğinde reddedilir.
- Device token API-1'de üretilir, imzalı istek API-2 ve API-3'te doğrulanır.
- Aynı jti+nonce iki node'a paralel gönderildiğinde yalnız biri başarılı olur.

3. P0-2 - SignalR Redis backplane

Program.cs 253-259 aralığında yalnız AddSignalR() kayıtlıdır. Projede Microsoft.AspNetCore.SignalR.StackExchangeRedis paketi ve AddStackExchangeRedis yapılandırması yoktur.

- ECSPros.Api.csproj dosyasına Microsoft.AspNetCore.SignalR.StackExchangeRedis paketi eklenmeli.
- AddSignalR(...).AddStackExchangeRedis(...) ile Redis state master-set/Sentinel bağlantısı kullanılmalı.
- ChannelPrefix ortam ve uygulama adına göre tanımlanmalı; staging ile production aynı pub/sub kanalını kullanmamalı.
- DashboardPresenceTracker process-local toplam sayıya güvenmemeli. Metrik worker her node'da yayınlamamalı veya liderlik kilidi kullanmalı.

Kabul testi

- Client API-1 SignalR hub'ına bağlanır; API-2 üzerinden üretilen NotificationHub mesajını alır.
- API-1 kapatılıp reconnect API-3'e geçtiğinde grup/abonelik uygulama protokolüyle yeniden kurulur.
- Redis state primary failover sırasında istemciler kontrollü reconnect olur ve uygulama process'i çökmez.

4. P0-3 - Data Protection ortak key ring

Program.cs 137-146, key ring'i varsayılan olarak ~/.ecsp/keys yerel dizinine yazar. CoreDbContext bu protector ile FirmPlatformIntegration.Credentials alanını şifreler ve çözer.

- PersistKeysToFileSystem yerel yol kullanımı production'da kaldırılmalı.
- Tercih: PostgreSQL-backed Data Protection key repository veya bütün API'lerin aynı anda gördüğü güvenilir shared storage.
- Bütün node'larda SetApplicationName("ECSPros") ve aynı key encryption certificate/KEK kullanılmalı.
- Mevcut key ring migration sırasında kaybedilmemeli; aksi halde mevcut integration credentials çözülemez.
- Key rotation ve eski key ile decrypt senaryosu çapraz node test edilmelidir.

5. P0-4 - Yerel dosya kullanımını kaldır

Alan	Kanıt	Gerekli hedef
Catalog görsel/video	Catalog Infrastructure DependencyInjection.cs 23-24 LocalDisk servislerini seçiyor	IObjectStorage / CDN adapter
Vitrin medya	PagesController.cs 181-205	Ortak media storage
İade görselleri	StoreAccountController.cs 295-305	Ortak media storage
Yorum görselleri	StoreReviewsController.cs 143-153	Ortak media storage
Talep ekleri	RequestsController.cs 138-145	Private object storage + signed URL
Feed	FeedGeneratorWorker.cs 19-40, 113-160	Object storage + DB/Redis status

- IFileStorage altında PutAsync, DeleteAsync, OpenReadAsync ve URL üretimi tek sözleşmeye alınmalı.
- Public ürün/vitrin medyası CDN origin storage'a; özel talep ekleri private bucket'a yazılmalı.
- Controller'lar fiziksel Path veya File.Create kullanmamalı.
- FeedTrigger process-local Channel yerine Redis stream veya PostgreSQL job tablosu kullanılmalı; feed status DB/Redis'te tutulmalı.
- Eski /media URL'leri için migration/copy ve geriye uyumlu URL yönlendirmesi hazırlanmalı.

6. P0-5 - Worker liderliği ve atomic outbox claim

Program.cs 282 ve 312-351 aralığında çok sayıda AddHostedService kaydı her API process'inde başlar. Integration Infrastructure DependencyInjection.cs ayrıca MarketplaceOrderFetchWorker'ı her instance'ta ekler.

Worker	Mevcut risk	Zorunlu koruma
TrackingDispatchWorker	Aynı pending event dış servise birden çok kez gönderilebilir	FOR UPDATE SKIP LOCKED + processing lease + dedup id
CargoNotifyWorker	Aynı kargo API çağrısı tekrarlanabilir	Atomic claim + carrier idempotency key
SavedSearchNotifyWorker	Aynı e-posta birden çok gönderilebilir	Atomic claim + notification unique key
SettlementEligibilityWorker	Aynı hakediş birden fazla cari hareket oluşturabilir	Row lock + unique ReferenceType/Referenceld
MarketplaceBatchWorker	Aynı batch paralel poll edilir, sayaçlar yarışır	Lease/row lock
LegacySyncWorker	Üç node eski MySQL'i aynı anda tarar	Per-slice advisory lock veya tek worker role
FeedGeneratorWorker	Üç node aynı feed'i aynı diske üretir	Distributed job + ortak storage

Önerilen desen

```
BEGIN; SELECT ... FROM outbox WHERE status='pending' AND next_attempt_at<=now() ORDER BY created_at FOR UPDATE SKIP LOCKED LIMIT 50; UPDATE ... SET status='processing', lease_owner=@node, lease_until=now()+interval '2 minutes'; COMMIT;
```

- Her dış sistem çağrısı için kalıcı idempotency key tutulmalı. Sadece process lock yeterli değildir.
- Lease süresi dolan processing kayıtları tekrar sahiplenilebilir olmalı.
- BackgroundServiceExceptionBehavior.Ignore korunacaksa her worker heartbeat ve son hata metriği üretmeli; sessiz ölüm kabul edilmemeli.
- Geçiş döneminde NodeRole=Api|Worker config gate eklenerek worker'lar yalnız seçili instance'ta çalıştırılmalı. Bu geçici korumadır; kalıcı HA için distributed claim gerekir.

7. P1 - Login ve rate limit state

LoginMemberCommand.cs 30-52, hatalı giriş sayacını IMemoryCache'te tutar ve yorumda açıkça tek host için tasarlandığını belirtir. Program.cs 433-478 içindeki ASP.NET rate limiter da process-local çalışır.

- Hesap bazlı login sayacı Redis INCR + EXPIRE ile atomik yapılmalı.
- IP bazlı ana hacim limiti Nginx shared zone'da uygulanmalı; uygulama limiti ikinci savunma olabilir.
- CF-Connecting-IP yalnız güvenilir Cloudflare/Nginx proxy zincirinden kabul edilmeli; dış istemci bu header'ı doğrudan enjekte edememeli.
- ASP.NET ForwardedHeaders middleware trusted proxy/network listesiyle etkinleştirilmeli.

8. P1 - PostgreSQL multi-host bağlantı

Program.cs 81-88 tek DefaultConnection ile NpgsqlDataSourceBuilder.Build() kullanır. PostgreSQL promotion sonrasında uygulamanın yeni primary'yi seçebilmesi için multi-host datasource gerekir.

```
Host=pg-1.internal,pg-2.internal;Port=5432;Database=ecspros;Target Session Attributes=primary;Host Recheck Seconds=3;Maximum Pool Size=50;
```

- NpgsqlDataSourceBuilder.BuildMultiHost() kullanılmalı ve primary target session seçilmeli.
- Bütün DbContext'ler ve raw Npgsql erişimleri aynı multi-host datasource üzerinden beslenmeli.
- Health check yalnız bağlantı değil SELECT 1 ve primary/read-write kontrolü yapmalı.
- Promotion sırasında kırılan açık transaction otomatik başka sunucuda devam etmez. Güvenli command boundary'lerde execution strategy/idempotent retry uygulanmalı.

9. Health endpoint ayrımı

Mevcut /health PostgreSQL'i zorunlu, Redis'i degraded ve HTTP 200 kabul ediyor. Redis state ve SignalR için zorunlu hale geldiğinde bu davranış production readiness için yetersizdir.

Endpoint	Kontrol	Başarısızlık
/live	Process event loop cevap veriyor	Process restart
/ready	PostgreSQL primary/read-write + Redis state + kritik config	Load balancer rotasyonundan çıkar
/health/detail	DB, Redis cache/state, storage, worker heartbeat; korumalı	Monitoring alarmı

10. Config ve secret sözleşmesi

Ayar	Kural
Jwt:Secret / Issuer / Audience	Bütün API node'larında birebir aynı; secret manager'dan alınır.
DataProtection	Ortak repository, aynı ApplicationName, key encryption etkin.
Redis	Sentinel endpoint listesi, master set adı, cache/state ayrımı.
PostgreSQL	İki host, primary target, pool limitleri, TLS doğrulama.
Node identity	NodeId ve NodeRole her VM'de farklı/açık.
Storage	Bucket/container, public base URL, private signed URL ayarları.
Environment	Production ve staging Redis channel/key prefix'leri ayrık.

Secret değerleri appsettings repository'sine veya PDF'ye yazılmamalıdır. appsettings yalnız secret reference/placeholder içermelidir.

11. Önerilen uygulama fazları

Faz	Teslimat	Release ölçütü
A - State	Redis state store, login counter, nonce/challenge atomicity	Çapraz-node güvenlik testleri yeşil
B - Realtime	SignalR Redis backplane, presence/worker liderliği	Node'lar arası hub testleri yeşil
C - Crypto/storage	Ortak Data Protection, object/media storage	Cross-node decrypt ve upload/read testleri yeşil
D - Workers	Atomic claim, leases, unique idempotency	Paralel worker duplicate testi yeşil
E - Database HA	Npgsql multi-host, primary health, retry boundaries	Patroni switchover/failover testi yeşil
F - Release	4 API, Nginx health, load/failure test	Node kaybında SLO karşılanır

12. Zorunlu test matrisi

Test türü	Senaryo	Beklenen
Integration	API-1 create -> API-2 read/consume	State ve crypto bütün node'larda aynı
Concurrency	Aynı nonce/outbox satırına 100 paralel çağrı	Yalnız bir başarılı tüketim
SignalR	Client node A, publish node B	Mesaj eksiksiz gelir
Storage	Upload node A, download node B/Nginx	Dosya ve metadata bulunur
Worker	Üç worker aynı anda başlar	Tek dış çağrı/tek ledger movement
PG failover	Primary kill	Yeni transaction yeni primary'de devam eder; duplicate yok
Redis failover	Master kill	Sentinel geçişi; state güvenlik politikası korunur
Load	4.000 aktif kullanıcı senaryosu	p95, error rate, pool ve IOPS eşikleri karşılanır

13. Değişiklik yapılacak ana dosyalar

Dosya / alan	Değişiklik
src/ECSPros.Api/Program.cs	SignalR backplane, multi-host Npgsql, shared DP, role-gated worker, /live-/ready, forwarded headers
Services/Store/DeviceAttestationServices.cs	IMemoryCache yerine IDeviceStateStore
Controllers/StoreDeviceController.cs	Atomic Redis challenge consume
Middleware/DeviceRequestGuardMiddleware.cs	Atomic Redis nonce consume
Crm LoginMemberCommand.cs	Redis distributed login counter
Tracking/Cargo/SavedSearch/Settlement workers	Claim lease, idempotency, row/advisory lock
Catalog Infrastructure DI ve LocalDisk servisleri	Production object storage adapter
Pages/StoreAccount/StoreReviews/Requests controllers	Storage abstraction
FeedGeneratorWorker.cs	Distributed trigger/status ve object output
appsettings şeması	Sentinel, PG hosts, NodeRole, storage, health ayarları

14. Tamamlanma tanımı

- Hiçbir güvenlik veya doğruluk state'i yalnız IMemoryCache'te tutulmuyor.
- Hiçbir public/private upload yalnız API local diskine yazılmıyor.
- Her dış etkili worker kaydı atomik sahipleniyor ve idempotency garantisi taşıyor.
- SignalR mesajları API node sınırını geçiyor.
- Data Protection key rotation tüm node'larda decrypt edilebiliyor.
- PostgreSQL ve Redis failover otomatik testte geçiyor.
- Bir fiziksel host kapalıyken 4.000 aktif kullanıcı yük testi belirlenen SLO'yu karşılıyor.

Son karar

Bu tamamlanma kriterlerinden biri eksikse sistem çoklu API production yayınına hazır kabul edilmemelidir.

15. Resmi teknik kaynaklar

- Npgsql Multi-host: <https://www.npgsql.org/doc/failover-and-load-balancing.html>
- Redis Sentinel: https://redis.io/docs/latest/operate/oss_and_stack/management/sentinel/
- Redis Replication: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/
- PostgreSQL Warm Standby: <https://www.postgresql.org/docs/17/warm-standby.html>
- Patroni: <https://patroni.readthedocs.io/en/latest/>
- Cloudflare Load Balancing Monitors: <https://developers.cloudflare.com/load-balancing/monitors/>